

Code of the paper:

Installation:

```
git clone git@github.com:AntonioTepsich/ckan.git
cd Convolutional-KANs
pip install -r requirements.txt
```

Usage:

```
from kan_convolutional.KANConv import KAN_Convolutional_Layer
```

Example to Construct a KANConv for MNIST:

```
import torch
from torch import nn
import torch.nn.functional as F

from kan_convolutional.KANConv import KAN_Convolutional_Layer

class KANC_MLP(nn.Module):
    def __init__(self, grid_size: int = 5):
        super().__init__()
        self.conv1 = KAN_Convolutional_Layer(in_channels=1,
                                             out_channels=5,
                                             kernel_size=(3,3),
                                             grid_size=grid_size
                                             )

        self.conv2 = KAN_Convolutional_Layer(in_channels=5,
                                             out_channels=5,
                                             kernel_size=(3,3),
                                             grid_size=grid_size
                                             )

        self.pool1 = nn.MaxPool2d(
            kernel_size=(2, 2)
        )

        self.flat = nn.Flatten()

        self.linear1 = nn.Linear(125, 10)
        self.name = f"KANC MLP (Small) (gs = {grid_size})"
```

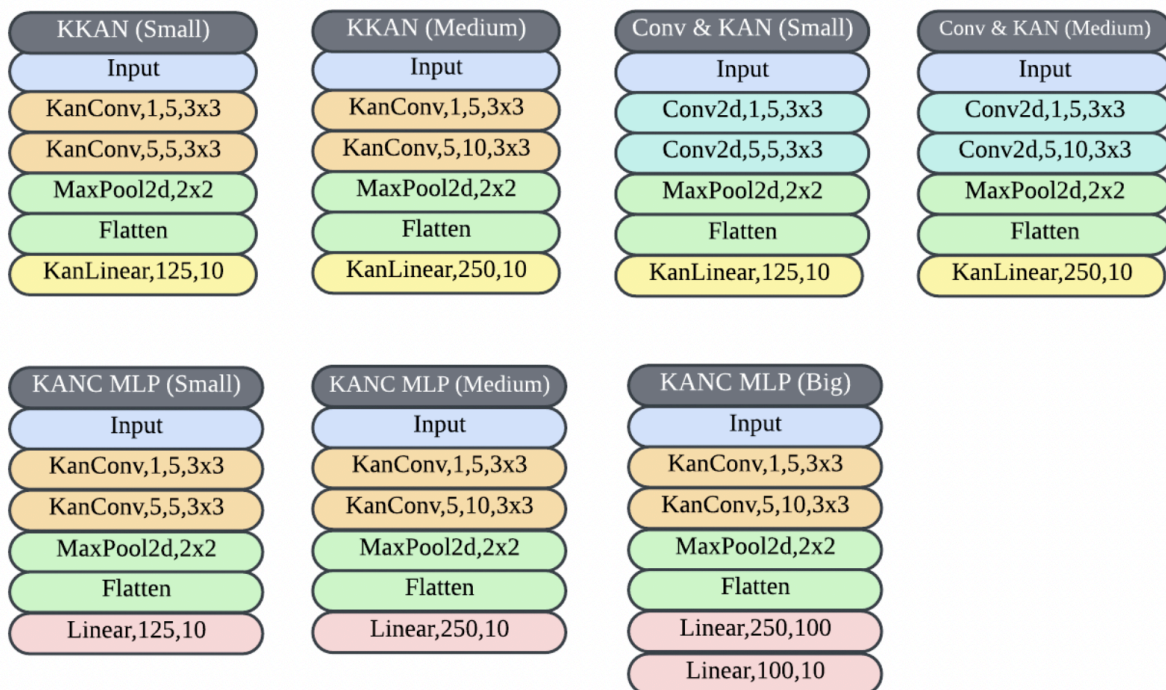
```
def forward(self, x):
    x = self.conv1(x)

    x = self.pool1(x)

    x = self.conv2(x)
    x = self.pool1(x)
    x = self.flat(x)
    #print(x.shape)
    x = self.linear1(x)
    x = F.log_softmax(x, dim=1)
    return x
```

<https://github.com/AntonioTepsich/Convolutional-KANs?tab=readme-ov-file>

KAN architectures



Theoretically:

The core of KANs resides in their unique architecture. Unlike traditional MLPs that use fixed activation functions at nodes, KANs implement learnable activation functions on the network edges. This critical shift from static to dynamic node functions involves replacing conventional linear weight matrices with adaptive spline functions, which are parametrized and optimized during training. This allows for a more flexible and responsive model architecture that can dynamically adapt to complex data patterns.

In more detail, the Kolmogorov-Arnold representation theorem posits that a continuous multivariate function $f(x_1, \dots, x_n)$ can be expressed as:

$$f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$$

Here, $\phi_{q,p}$ are univariate functions mapping each input variable (x_p) such $\phi_{q,p} : [0, 1] \rightarrow \mathbb{R}$, and $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$, univariate functions.

KANs structure each layer as a matrix of these learnable 1D functions:

$$\Phi = \{\phi_{q,p}\}, \quad p = 1, 2, \dots, n_{in}, \quad q = 1, 2, \dots, n_{out}$$

$$\text{spline}(x) = \sum_i c_i B_i(x), \quad c_i \text{ are trainable coefficients}$$

This formulation allows each $\phi_{l,j,i}$ to adapt its shape based on the data, offering unprecedented flexibility in how the network models interactions between inputs.

The overall structure of a KAN is analogous to stacking layers in MLPs, but with the enhancement of utilizing complex functional mappings instead of simple linear transformations and nonlinear activations:

$$\text{KAN}(x) = (\Phi_{L-1} \circ \Phi_{L-2} \circ \dots \circ \Phi_0)(x)$$

Each layer's transformation, Φ_l , acts on the input x_l to produce the next layer's input x_{l+1} , described as:

$$x_{l+1} = \Phi_l(x_l) = \begin{pmatrix} \phi_{l,1,1}(\cdot) & \cdots & \phi_{l,1,n_l}(\cdot) \\ \vdots & \ddots & \vdots \\ \phi_{l,n_{l+1},1}(\cdot) & \cdots & \phi_{l,n_{l+1},n_l}(\cdot) \end{pmatrix} x_l$$

where each activation function $\phi_{l,j,i}$ is a spline, providing a rich, adaptable response surface to model inputs.

In computer vision, convolutions are a foundational operation in Convolutional Neural Networks (CNNs), where a kernel or filter slides **across the input, computing dot products at each position to extract spatial features**. Unlike traditional CNNs, where kernels consist of fixed weights, KAN Convolutions use kernels composed of learnable non-linear functions, denoted by ϕ , which are parameterized using B-splines. This approach allows each kernel element to adapt dynamically during training, enabling greater flexibility and expressiveness. Formally, each element of the kernel is defined as:

$$\phi = w_1 \cdot \text{spline}(x) + w_2 \cdot \text{silu}(x) \quad (1)$$

In a KAN Convolution, the kernel slides over the image and applies the corresponding activation function, ϕ_{ij} to the corresponding pixel, a_{kl} and calculates the output pixel as the sum of $\phi_{ij}(a_{kl})$. Let K be a KAN kernel $\in \mathbb{R}^{N \times M}$, and an image as a matrix as:

$$\text{Image} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1p} \\ a_{21} & a_{22} & \cdots & a_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mp} \end{bmatrix} \quad (2)$$

Then a KAN Convolutions is defined as follows in the Equation 3

$$(\text{Image} * K)_{i,j} = \sum_{k=1}^N \sum_{l=1}^M \phi_{kl}(a_{i+k,j+l}) \quad (3)$$

$$\begin{aligned} \text{KAN Kernel} &= \begin{bmatrix} \phi_{11} & \phi_{12} \\ \phi_{21} & \phi_{22} \end{bmatrix} \\ \text{Image} * \text{KAN Kernel} &= \begin{bmatrix} \phi_{11}(a_{11}) + \phi_{12}(a_{12}) + \phi_{21}(a_{21}) + \phi_{22}(a_{22}) & \cdots & r_{1(p-1)} \\ \phi_{11}(a_{21}) + \phi_{12}(a_{22}) + \phi_{21}(a_{31}) + \phi_{22}(a_{32}) & \cdots & r_{2(p-1)} \\ \vdots & \ddots & \vdots \\ \phi_{11}(a_{m1}) + \phi_{12}(a_{m2}) + \phi_{21}(a_{(m+1)1}) + \phi_{22}(a_{(m+1)2}) & \cdots & r_{m(p-1)} \end{bmatrix} \end{aligned} \quad (4)$$

So in comparison:

$$\text{Kernel} = \begin{bmatrix} k_{11} & k_{12} \\ k_{21} & k_{22} \end{bmatrix}$$

(a) Kernel

$$\text{KAN Kernel} = \begin{bmatrix} \phi_{11} & \phi_{12} \\ \phi_{21} & \phi_{22} \end{bmatrix}$$

(b) KAN Kernel

$$\text{Image} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1p} \\ a_{21} & a_{22} & \cdots & a_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mp} \end{bmatrix}$$

$$\begin{bmatrix} k_{11} \cdot a_{11} + k_{12} \cdot a_{12} + k_{21} \cdot a_{21} + k_{22} \cdot a_{22} & \cdots & r_{1(p-1)} \\ k_{11} \cdot a_{21} + k_{12} \cdot a_{22} + k_{21} \cdot a_{31} + k_{22} \cdot a_{32} & \cdots & r_{2(p-1)} \\ \vdots & \vdots & \ddots & \vdots \end{bmatrix}$$

(a) Result of Classic Convolution

$$\begin{bmatrix} \phi_{11}(a_{11}) + \phi_{12}(a_{12}) + \phi_{21}(a_{21}) + \phi_{22}(a_{22}) & \cdots & r_{1(p-1)} \\ \phi_{11}(a_{21}) + \phi_{12}(a_{22}) + \phi_{21}(a_{31}) + \phi_{22}(a_{32}) & \cdots & r_{2(p-1)} \\ \vdots & \vdots & \ddots & \vdots \end{bmatrix}$$

(b) Result of KAN Convolution

Networks:

CKAN_SMLM v1

(B, 1, 7, 7) → KAN_Conv(1→8) → (B, 8, 7, 7)
→ KAN_Conv2(8→16) → (B, 16, 7, 7)
→ MaxPool(2×2) → (B, 16, 3, 3)
→ Flatten → (B, 144)
→ KAN_Linear(144→32)
→ KAN_Linear(32→4)
→ (B, 2, 2)

CKAN_SMLM v2

(B, 1, 7, 7)
↓
KAN_Conv(1→16) + BatchNorm → (B, 16, 7, 7)
KAN_Conv(16→32) + BatchNorm → (B, 32, 7, 7)
KAN_Conv(32→32) + BatchNorm → (B, 32, 7, 7)
↓
Flatten → (B, 1568)
↓
KAN_Linear(1568→128)
KAN_Linear(128→64)
KAN_Linear(64→4)
↓
(B, 2, 2)

Property	V1	V2
Input	(B, 1, 7, 7)	(B, 1, 7, 7)
Output	(B, 2, 2)	(B, 2, 2)
Total Parameters	~15K	~180K
Grid Size	5	8
Spline Order	3	3
Best Val RMSE	~50 nm	~35 nm

V1 architecture:

Layer	Output Shape
Input	(B, 1, 7, 7)
KAN_Conv2d(1→8, 3×3, pad=1)	(B, 8, 7, 7)
KAN_Conv2d(8→16, 3×3, pad=1)	(B, 16, 7, 7)
MaxPool2d(2×2)	(B, 16, 3, 3)
Flatten	(B, 144)
KAN_Linear(144→32)	(B, 32)
KAN_Linear(32→4)	(B, 4)
Reshape	(B, 2, 2)

V1 Training Config

Parameter	Value
Batch Size	32
Epochs	100
Learning Rate	1e-3
Optimizer	Adam
Convo layers	3
Grid size	5
Loss Function	Wing Loss

V2 Architecture

Layer	Output Shape	
Input	(B, 1, 7, 7)	-

KAN_Conv2d(1→16, 3×3, pad=1)	(B, 16, 7, 7)	
BatchNorm2d(16)	(B, 16, 7, 7)	

KAN_Conv2d(16→32, 3×3, pad=1)	(B, 32, 7, 7)	
BatchNorm2d(32)	(B, 32, 7, 7)	

KAN_Conv2d(32→32, 3×3, pad=1)	(B, 32, 7, 7)	
BatchNorm2d(32)	(B, 32, 7, 7)	

Flatten	(B, 1568)	-

KAN_Linear(1568→128)	(B, 128)	
Dropout(0.2)	(B, 128)	

KAN_Linear(128→64)	(B, 64)	

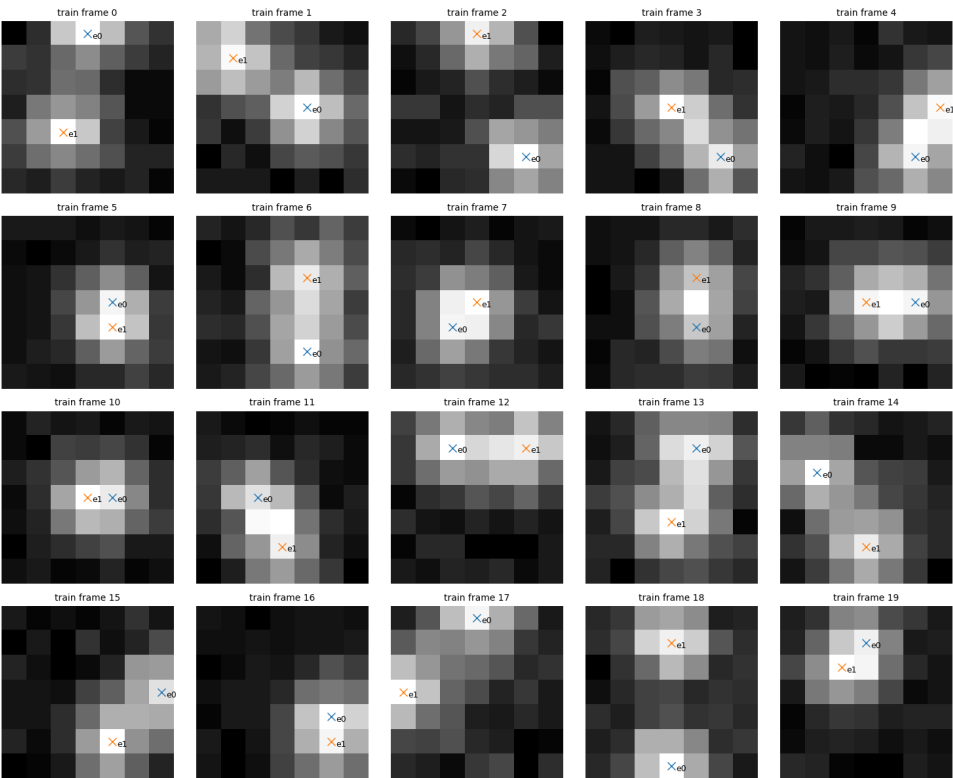
KAN_Linear(64→4)	(B, 4)	

Reshape	(B, 2, 2)	-
=====		

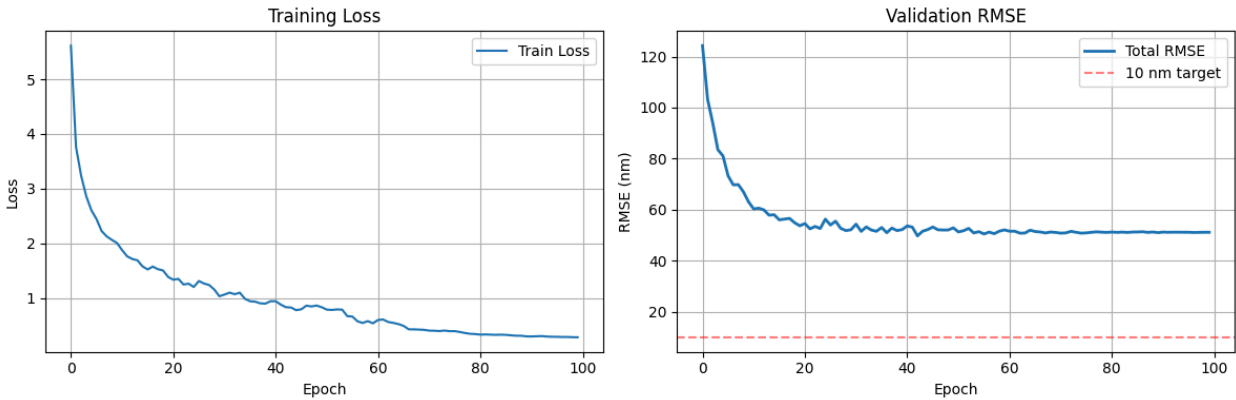
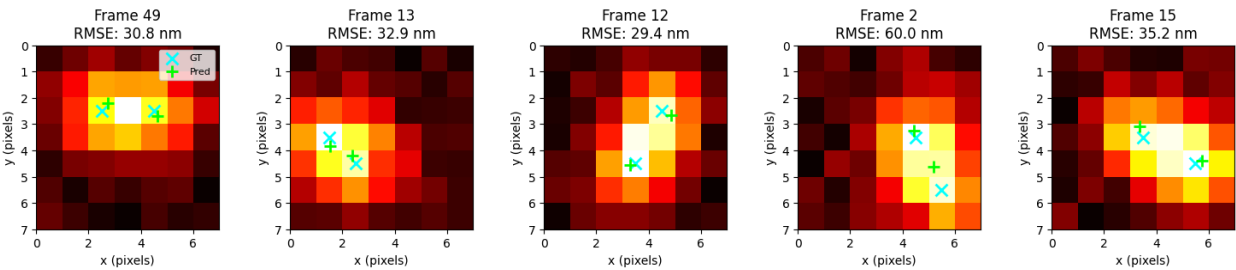
V2 Training Config

Parameter	Value
Batch Size	64
Epochs	150
Learning Rate	3e-3
Optimizer	Adam
Loss Function	Wing Loss
Convo layers	3
Grid size	8
Gradient Clipping	Yes

Dataset 7*7 (emitters in the center of the pixels) 2 emitters per frame



Result network version1:



Result network version2:

